

Ticket #001: Server experiencing high CPU load and system slowdown.

Monitoring alerts show CPU usage at an all-time high.

Symptoms: CPU usage steadily climbs, system load is high, server responds slowly to requests, applications lag or time out, users report degraded performance

Resolution:

1. Running top to verify the server's CPU spike and identify what processes are causing the high usage.

```
top - 15:32:26 up 28 min, 2 users, load average: 29.21, 24.68, 15.67
Tasks: 167 total, 29 running, 118 sleeping, 0 stopped, 20 zombie
%Cpu(s): 76.4 us, 23.6 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3709.1 total, 2170.6 free, 333.7 used, 1366.0 buff/cache
MiB Swap: 1893.0 total, 1893.0 free, 0.0 used, 3375.4 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
1067196 root        20   0  3128 1712 1124 R 29.2   0.0   0:00.88 gzip
1069507 root        20   0  3128 1724 1136 R 24.9   0.0   0:00.75 gzip
1071457 root        20   0  3128 1724 1136 R 22.3   0.0   0:00.67 gzip
  1066 jay         20   0  7048 3344 3080 R  1.0   0.1   1:24.79 faulty_log_comp
 231779 jay         20   0  7048 3324 3068 R  1.0   0.1   0:11.14 faulty_log_comp
 314533 jay         20   0  7048 3336 3080 R  1.0   0.1   0:01.33 faulty_log_comp
 624726 jay         20   0  7048 3348 3080 R  1.0   0.1   0:24.69 faulty_log_comp
```

Analysis: CPU usage is at 76.4%, which is unusual for this VM server. There are multiple processes constantly running from a mysterious file named “faulty_log_comp.” The CPU usage for each one is small, but the issue is constant process spawning. PIDs are continuously changing, indicating repeated process spawning and sustaining the CPU load.

2. Investigating further with the command ps -ef.

```
root      12503    1014  0 18:17 ?        00:00:00 /usr/sbin/CRON -f -P
jay        12505    12503  0 18:17 ?        00:00:00 /bin/sh -c /usr/local/bin/faulty_log_compress.sh
jay        12506    12505  0 18:17 ?        00:00:00 /bin/bash /usr/local/bin/faulty_log_compress.sh
jay        12508    12506  5 18:17 ?        00:00:03 /bin/bash /usr/local/bin/faulty_log_compress.sh
jay        12509    12506  5 18:17 ?        00:00:02 /bin/bash /usr/local/bin/faulty_log_compress.sh
jay        12510    12506  5 18:17 ?        00:00:02 /bin/bash /usr/local/bin/faulty_log_compress.sh
jay        12511    12506  5 18:17 ?        00:00:02 /bin/bash /usr/local/bin/faulty_log_compress.sh
root       318592   11175 23 18:17 pts/1    00:00:00 gzip -kf /var/log/faultlab/app2.log
root       318620   11174 24 18:17 pts/1    00:00:00 gzip -kf /var/log/faultlab/app2.log
root       319169   11173 23 18:17 pts/1    00:00:00 gzip -kf /var/log/faultlab/app2.log
root       319780   11176 23 18:17 pts/1    00:00:00 gzip -kf /var/log/faultlab/app2.log
```

Analysis: The process list shows multiple instances of faulty_log_compress.sh with differing PPIDs, indicating independent invocations rather than a single parent process spawning children. This pattern is consistent with a scheduled task triggering repeated executions.

3. Verifying the process tree with the command ps -ef --forest.

```
root      18558      1  0 Apr24 pts/0    00:00:00 sudo /usr/local/bin/faulty_log_compress.sh
root      18559    18558  0 Apr24 pts/1    00:00:00 \_ sudo /usr/local/bin/faulty_log_compress.sh
root      18560    18559  0 Apr24 pts/1    00:00:00 \_ /bin/bash /usr/local/bin/faulty_log_compress
root      18561    18560  0 Apr24 pts/1    00:00:00 \_ /bin/bash /usr/local/bin/faulty_log_com
root     1748568    18561 24 00:28 pts/1    00:00:00 | \_ gzip -kf /var/log/faultlab/app2.log
root      18562    18560  0 Apr24 pts/1    00:00:00 \_ /bin/bash /usr/local/bin/faulty_log_com
root     1751961    18562 25 00:28 pts/1    00:00:00 | \_ gzip -kf /var/log/faultlab/app1.log
root      18563    18560  0 Apr24 pts/1    00:00:00 \_ /bin/bash /usr/local/bin/faulty_log_com
root     1748404    18563 23 00:28 pts/1    00:00:00 | \_ gzip -kf /var/log/faultlab/app2.log
root      18565    18560  0 Apr24 pts/1    00:00:00 \_ /bin/bash /usr/local/bin/faulty_log_com
root     1749260    18565 23 00:28 pts/1    00:00:00 \_ gzip -kf /var/log/faultlab/app2.log
```

Analysis: The repeating executions stem from faulty_log_compress.sh which triggers multiple gzip compression operations on log files.

4. Inspecting faulty_log_compress.sh.

```
[jay@mainserver:~$ cat /usr/local/bin/faulty_log_compress.sh
#!/bin/bash

while true
do
    for file in /var/log/faultlab/*.log
    do
        gzip -kf "$file"
    done
done
```

Analysis: The “while true” loop creates an infinite execution cycle, which explains the repeated gzip compression operations on log files.

5. Killing the faulty script and removing it. Verifying via top that no processes remain.

```
[jay@mainserver:~$ sudo kill -f faulty_log_compress.sh
[[sudo] password for jay:
```

```
[jay@mainserver:~$ sudo rm /usr/local/bin/faulty_log_compress.sh
```

```
Tasks: 123 total, 8 running, 113 sleeping, 0 stopped, 2 zombie
%Cpu(s): 23.3 us, 76.7 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 3709.1 total, 2682.8 free, 331.8 used, 853.4 buff/cache
MiB Swap: 1893.0 total, 1893.0 free, 0.0 used, 3377.3 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
875152	jay	20	0	7048	3336	3080	R	5.0	0.1	0:00.32	faulty_log_comp
261150	jay	20	0	7048	3340	3080	S	4.7	0.1	0:03.44	faulty_log_comp
396408	jay	20	0	7048	3332	3076	S	4.7	0.1	0:55.62	faulty_log_comp
738519	jay	20	0	7048	3328	3072	S	4.7	0.1	0:41.42	faulty_log_comp
1412158	jay	20	0	7048	3336	3080	S	4.7	0.1	0:29.82	faulty_log_comp
2042865	jay	20	0	7048	3336	3080	R	4.7	0.1	0:21.74	faulty_log_comp
2630355	jay	20	0	7048	3332	3076	R	4.7	0.1	0:16.08	faulty_log_comp
3818283	jay	20	0	7048	3340	3080	R	4.7	0.1	0:06.93	faulty_log_comp
3203826	jay	20	0	7048	3340	3080	S	4.3	0.1	0:11.26	faulty_log_comp
24	root	20	0	0	0	0	S	1.7	0.0	1:07.53	ksoftirqd/1

Analysis: There are still a few leftover processes running. This persistence points to a scheduled execution.

6. Inspecting the cron configuration.

```
[jay@mainserver:~$ crontab -l
* * * * * /usr/local/bin/faulty_log_compress.sh
```

Analysis: The faulty script is configured in cron to execute every minute, resulting in running continuous execution. Even with the script deleted from disk, any instances already loaded in RAM would continue until terminated, and cron continues executing the job every minute, leading to repeated process spawning.

7. Opening the crontab editor and commenting out the faulty entry.

```
[jay@mainserver:~$ sudo crontab -e

Select an editor. To change later, run 'select-editor'.
 1. /bin/nano        <---- easiest
 2. /usr/bin/vim.basic
 3. /usr/bin/vim.tiny
 4. /bin/ed

Choose 1-4 [1]: 1
crontab: installing new crontab
```

```
GNU nano 7.2 /tmp/crontab.ZRJrWh/crontab *
# * * * * * /usr/local/bin/faulty_log_compress.sh
```

8. Confirming the faulty crontab entry is properly commented out.

```
[jay@mainserver:~$ sudo crontab -l
#* * * * * /usr/local/bin/faulty_log_compress.sh
```

9. Terminating the leftover processes.

```
[jay@mainserver:~$ sudo pkill -f faulty_log_compress.sh
[sudo] password for jay:
jay@mainserver:~$
```

10. Running the top command and ps -ef command to confirm no active processes remain in memory and CPU usage returns to normal.

```
top - 21:49:35 up 3:46, 2 users, load average: 0.00, 1.17, 6.13
Tasks: 106 total, 1 running, 105 sleeping, 0 stopped, 0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 3769.1 total, 2904.1 free, 351.1 used, 612.9 buff/cache
MiB Swap: 1893.0 total, 1893.0 free,  0.0 used, 3358.0 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
3121285 jay       20   0   11768   5264   3148 R   9.1   0.1   0:00.01 top
  1 root      20   0   21848  12372   8668 S   0.0   0.3   0:00.85 systemd
  2 root      20   0      0      0      0 S   0.0   0.0   0:00.00 kthreadd
  3 root      20   0      0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
  4 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
  5 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
  6 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_
  7 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
  9 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
 12 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
 13 root      20   0      0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
 14 root      20   0      0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
 15 root      20   0      0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
 16 root      20   0      0      0      0 S   0.0   0.0   2:00.17 ksoftirqd/0
 17 root      20   0      0      0      0 I   0.0   0.0   0:39.54 rcu_preempt
 18 root      rt   0      0      0      0 S   0.0   0.0   0:01.61 migration/0
 19 root     -51   0      0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
 20 root      20   0      0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
 21 root      20   0      0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
 22 root     -51   0      0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
 23 root      rt   0      0      0      0 S   0.0   0.0   0:01.57 migration/1
 24 root      20   0      0      0      0 S   0.0   0.0   1:52.41 ksoftirqd/1
 26 root      0 -20      0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri

jay@mainserver:~$ ps -ef | grep faulty_log_comp
jay      3121287 1226648  0 21:49 pts/4    00:00:00 grep  --color=auto faulty_log_comp
jay@mainserver:~$
```

Analysis: Success. Issue fully resolved.

11. Deleting the log files that the faulty script created.

```
[jay@mainserver:~$ ls /var/log/faultlab/
app1.log app2.log
[jay@mainserver:~$ sudo rm -r /var/log/faultlab
[jay@mainserver:~$ ls /var/log/faultlab/
ls: cannot access '/var/log/faultlab/': No such file or directory
```

Conclusion: The system performance issue was traced to faulty_log_compress.sh that continuously spawned gzip processes, leading to excessive CPU utilization. Process inspection revealed multiple compression tasks originating from the same parent script. After manual intervention and removal of the script, the processes kept running indicating a scheduled task from a scheduled cron job. Remediation involved commenting out the cron job and killing the active processes. This restored normal system performance and eliminated recurring process spikes.